PSoC 6 CapSense Pong — Project Plan (2-Player)

Team: Amir + partner **Duration:** ~1 night (3–4 hours) **Boards:** 2x CY8CPROTO-062-4343W (PSoC 6 Wi-Fi BT Prototyping Kit)

---

1. Concept

A classic Nokia-style two-player Pong game running on a laptop, with **each player holding their own PSoC 6 board** and controlling a paddle via the CapSense touch slider. One board also works (vs. AI), and the game runs keyboard-only for testing.

```
Player 1: slider → Board 1 → UART "P42\n" → COM5 -+
                                                  +→ pong.py (pygame)
Player 2: slider → Board 2 → UART "P73\n" → COM7 -+
```

KEY SIMPLIFICATION: both boards run **identical firmware**. They are distinguished purely by which COM port they enumerate as — no code differences, flash the same project onto both.

Why it's a good demo: CapSense (the board's signature feature), UART/retarget-io, real-time input, multi-device integration — all with zero extra hardware.

---

2. Requirements

Hardware

- 2x CY8CPROTO-062-4343W boards
- 2x USB Type-A to Micro-B **data** cables (one comes with each kit; test both — charge-only cables silently fail)
- Laptop with 2 free USB ports (or a hub), Windows, already set up

Software (already installed on Amir's laptop)

- ModusToolbox 3.5 (tools at `C:\Infineon\Tools\ModusToolbox\tools_3.5`)
- Eclipse IDE for ModusToolbox 3.4
- GCC 11.3.1 (`CY_COMPILER_GCC_ARM_DIR = C:/Infineon/Tools/mtb-gcc-arm-eabi/11.3.1/gcc`)
- Python 3 with: `pip install pygame pyserial`
- Game file: `pong.py` (provided)

Environment notes (important — hard-won lessons)

- Projects MUST live in a path without spaces: use `C:\Users\Public\Infineon\mtw`
- Env vars already set: `CY_TOOLS_PATHS`, `CY_COMPILER_GCC_ARM_DIR`
- Only ONE program can hold the COM port at a time (close Tera Term before running the game)

---

3. Firmware side (Person A)

3.1 Create the project

1. Open Eclipse → Quick Panel → **New Application**

2. BSP: **CY8CPROTO-062-4343W**
3. Template: **Peripherals → CapSense Buttons and Slider** (FreeRTOS or bare-metal version — either works)
4. Application root path: `C:\Users\Public\Infineon\mtw`
5. Create, wait for libraries to download

3.2 Modify the code

The example already scans the slider and has retarget-io (printf → UART) set up. Find the widget-processing part of the main loop and add:

```
/* After Cy_CapSense_ProcessAllWidgets() has run: */
if (Cy_CapSense_IsWidgetActive(CY_CAPSENSE_LINEARSLIDER0_WDGT_ID,
                               &cy_capsense_context))
{
    cy_stc_capsense_touch_t *touch =
        Cy_CapSense_GetTouchInfo(CY_CAPSENSE_LINEARSLIDER0_WDGT_ID,
                                 &cy_capsense_context);

    if (touch->numPosition > 0)
    {
        uint16_t pos = touch->ptrPosition->x;      /* 0 .. xResolution-1 */
        /* Slider resolution is 300 by default in this example.
           Check cycfg_capsense.h if paddle range looks wrong. */
        printf("P%u\n", (pos * 100u) / 300u);      /* scale to 0-100 */
    }
}
```

Notes: - Exact symbol names can differ slightly between example versions. If the example already printfs the slider position, just change its format string to `"P%u\n"`. - Keep the UART at **115200 baud** (the example default). - Send only when the slider is touched — the game holds the last position when you lift your finger.

3.3 Flash

1. Plug the board in (KitProg3 micro-B connector at the board edge)
2. Quick Panel → **Program (KitProg3_MiniProg4)**
3. Optional check: open Tera Term on the KitProg3 COM port @115200, touch the slider — you should see `P0`…`P100` streaming. **Close Tera Term afterwards.**

3.4 One-time housekeeping (already flagged during setup)

Update KitProg3 firmware (board plugged in):

```
C:\Infineon\Tools\ModusToolboxProgtools-1.3.1\fw-loader\bin\fw-loader.exe --update-kp3
```

---

4. PC / game side (Person B)

4.1 Setup

```
pip install pygame pyserial
```

4.2 Find the COM ports

Plug in ONE board first → Device Manager → Ports (COM & LPT) → **KitProg3 USB-UART** → note the COM number (e.g. COM5). Then plug in the second board and note the new port that appears (e.g. COM7). Label the boards with tape so you know which is which.

4.3 Run

```
python pong.py COM5 COM7    # two boards: P1 = first port, P2 = second
python pong.py COM5         # one board vs AI
python pong.py              # keyboard test (P1: W/S, P2: arrows)
```

If either serial port fails, that player automatically falls back to keyboard, so a loose cable doesn't kill the demo.

4.4 How pong.py works (for the report/presentation)

- A background thread reads the serial port line-by-line; lines starting with `P` update a shared `slider_pos` variable (0–100).
- The pygame loop (60 FPS) maps `slider_pos` to the left paddle's Y.
- Ball physics: speed increases 5% per paddle hit; deflection angle depends on where the ball strikes the paddle.
- AI paddle follows the ball with capped speed (beatable).
- Nokia-LCD colour palette: background `(155,188,15)`, foreground `(15,56,15)`.

---

5. Suggested task split

Task	Owner	Time
Create CapSense project, verify slider prints to UART	A	1–1.5 h
Test pong.py in keyboard mode, tune speeds/sizes	B	0.5 h
Integration: board + game, map feel (scaling, smoothing)	Both	0.5–1 h
Polish: score sound, start screen, README/demo video	Both	0.5 h

---

6. Troubleshooting

Symptom	Fix
`Serial error: could not open port`	Wrong COM number, or Tera Term still open
Paddle doesn't move, game says SLIDER	Firmware not printing `P<n>\n` — check in Tera Term first
Paddle jitters	Add smoothing: `y = 0.7*y_old + 0.3*y_new` in pong.py, or increase CapSense filter in the configurator
Paddle only covers part of screen	Slider resolution isn't 300 — check `cycfg_capsense.h`, adjust the scale divisor
Build errors about tools/GCC	Env vars: `CY_TOOLS_PATHS`, `CY_COMPILER_GCC_ARM_DIR` (values in §2)

Symptom	Fix
Programming fails: “no CMSIS-DAP device”	Board unplugged / charge-only cable / different USB port

---

7. Stretch goals (if time remains)

- **CapSense buttons** = start/pause.
- **Board LED feedback** — blink user LED on every paddle hit (PC sends a byte back; firmware reads UART and toggles LED).
- **Wi-Fi version** — send positions over UDP instead of UART (reuses the workshop’s Wi-Fi UDP example; wireless controller!).
- **Sound** — pygame beep on bounce/score for the full retro feel.